

AI as an Engineering Subject: A Theoretical Framework for Integrating Large Language Models into Intelligent Transportation Systems

Andrey A. Kotov, Pavel I. Pospelov

Moscow Automobile and Road Construction State Technical University (MADI)
Moscow, Russia
gt@madi.ru

Abstract—Road engineering practice is undergoing a marked shift with the deployment of large language models (LLMs) and related tool-based approaches for working with data and applied systems. It substantiates a shift from traditional programming skills to “meaning management” competencies: formalizing requirements and ensuring quality control during natural-language interaction. At a practical level, the integration of AI into road design and operation workflows is analyzed, including accelerating engineering calculations, managing parametric CAD models, and automating operations in software environments. The synergy of LLMs with GIS and computer vision for traffic-flow monitoring and report preparation is considered. Implementation barriers are assessed, including limitations of CAD geometric kernels and the economic component (inference cost and dependence on context length). It is concluded that AI is evolving into a management layer connecting knowledge, data, and tools, which increases requirements for engineering competencies in task formulation, specification, validation, and the safe operation of machine-generated solutions.

Keywords—*artificial intelligence analysis; large language models (LLMs); road engineering; parametric design; highways; knowledge management; CAD; geographic information systems (GIS); computer vision, data security; automation of calculations; AI economics*

I. EVOLUTION AND EXPECTATIONS: WHY 2022 BECAME A POINT OF NO RETURN

The rapid takeoff of chatbot systems based on large language models created a demand for inflated expectations in the real economy: that a model could, like a “plug-and-play expert,” deliver correct design solutions, automatically produce technical documentation, work with standards without errors, replace a development team when integrating with IT systems, and reliably make data-driven management decisions – in other words, produce industrial-quality output from a single prompt with minimal quality control. It is important, however, not to confuse a natural interface with a mature technology: conversational use went mainstream not because “conversation appeared,” but because responses stopped looking like rigid templates and increasingly resembled coherent reasoning and actionable instructions.

The public research release of ChatGPT in 2022 became a symbol of this shift, but the history of AI architecture research spans more than 100 years, with universities and R&D laboratories as the primary drivers of these technologies. The

qualitative leap that enabled modern-scale language models is associated with the Transformer architecture proposed in 2017 [1]. After that, stable scaling became possible, along with a transition from isolated “smart features” to a universal language interface capable of translating human intent into formal actions.

II. FROM EVERYDAY USE TO SPECIALIZED PRACTICE

From this point, it makes sense to talk not about aspirations but about practices – applications already validated by time, outcomes, cost, and quality.

The most visible everyday practice is lowering the barrier to programming while raising the bar for precise task formulation. Using modern systems no longer requires narrow expertise in adjacent IT fields, but an engineer still needs to understand the algorithm’s logic to validate results. Part of what used to require “tool mastery” now requires “meaning mastery.” Many people use this to prototype “here and now,” and the term “vibe coding” has emerged as a cultural marker: describing the goal in words and validating the result through system behavior while barely reading the source code [2].

Natural interaction in a unified engineering workspace is not limited to typed prompts. In global practice (Microsoft Copilot, integrations in AutoCAD), voice has moved beyond being a novelty and has become part of the user experience (UX). If a text model is the “brain,” then speech recognition technologies (STT/ASR) are the “ears,” letting an engineer remain in context without switching attention to typing. Transcribing discussions for international projects, lectures, interviews, and negotiations became widely accessible in the deep learning (DL) era, and modern toolchains allow choices between open-source and proprietary models. Architectures such as OpenAI Whisper [3] and the privacy-oriented IBM Granite Speech 3.3 have reached an accuracy where voice becomes a full-fledged design tool.

In professional work, however, quality still depends on background noise, accents, terminology, and recording conditions. A sound engineering approach is to validate quality on one’s own data and to keep the original audio together with an edit log if the transcript is used as a formal document. In engineering practice, the shift is not “model magic,” but several connected changes that can be captured as short key points:

- Role shift: Coding skills move to the background, while clear, consistent task specification becomes primary.
- Workflow mechanics: The engineer does not “program” the system as much as “dictate intent.”
- Interpretation: LLMs translate natural language into sequences of user-hidden actions and output a finished result (e. g., instant creation of a web resource in a local model).
- Countering “verbosity”: If a theory can be turned into Python code in minutes and checked by computation, empty talk becomes easier to expose.

A pragmatic question follows: what does this have to do with roads, and why does a transportation engineer need programming? Design, operation, and management of road infrastructure increasingly rely on digital artifacts – databases, models, geospatial layers, reports, automated checks, and the integration of sensors and video monitoring. Even when an engineer does not write code “for code’s sake,” they increasingly manage processes where code is the means of connecting computation, data, and requirements. In the road domain, the point is not the chatbot trend but the ability to turn a formalized idea into an executable procedure and verify it.

The most specialized practice in this logic is agent-based solutions that compute road indicators from data – because this is where verifiability becomes a direct requirement (Fig. 1). Any theory with a computational core can be encoded and reproducibly tested on examples; in education, a student sees that a polished explanation is not the same as a working algorithm. In engineering, it is even more critical: the computation must pass unit tests, edge cases, and agreement with a reference.

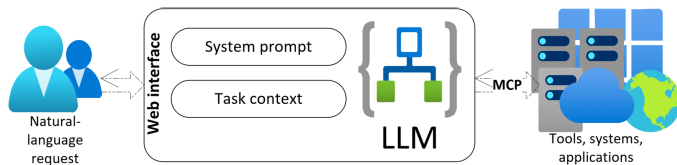


Fig. 1. AI agent diagram (architecture, validation, DB loop)

A highly promising practice is when code governs not only typical artifacts like web pages and “ordinary” software, but starts controlling geometry and drawings. In practice, this means: a person expresses requirements in words, the system generates a command sequence for the design environment, and the output becomes a drawing or a parametric model [4]. The “text-to-CAD sequence” idea is already an active research subject; an example is CAD sequence generation using visual feedback (CADFusion, 2025) [5]. A key consequence is that engineers are gradually freed from non-creative work – manual drafting, routine topological edits, mechanical selection of primitive libraries – so attention can focus on project ideas and verifiable solutions rather than clerical work. A strict quality criterion is emerging: if the technical narrative and specifications allow the design solution to be reconstructed without “telepathy,” the project is at least developed comprehensively; if reconstruction is impossible, digitization merely accelerates incomplete outputs and shifts the problem downstream to review and construction. In project-practice

terms, three pillars are required for automation to be sustainable:

- Standards compliance control: The working prompt is governed by system instructions that constrain the network’s actions within standards.
- Future workflow: Instead of learning specific programming languages (Python, C#, JavaScript), an engineer states the task in words, and the system assembles the necessary pipeline.
- Geometric kernels: Visualizing parameters or polygons requires a geometric kernel; as CAD systems evolve, developers who provide full script-level control over graphic kernels for direct linkage to LLMs will endure.

It is crucial not to substitute standards compliance with the promise that “the model follows standards.” A correct engineering stance is different: compliance is achieved through a control architecture. An LLM can help form commands and explain requirements, but adherence to standards is ensured through constraints, checks, libraries of standard solutions, and expert review. The real “simplification of automation” is not that the engineer stops thinking, but that they stop spending weeks learning low-level interfaces and instead assemble an execution scenario for the required task (Fig. 2).

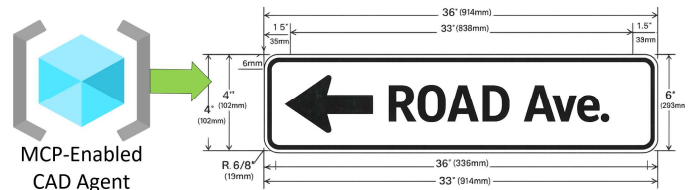


Fig. 2. Result of “text description –> CAD artifact” (road sign)

A logical next step is moving from flat drawings to “conversational” parametric control, where an engineer specifies a parameter change (“change the radius on the segment...,” “increase the design speed...”) and the system rebuilds the model. A scope of application should be set up front: fully automatic “design a road from A to B” as a complete project is not today’s task, because it depends not only on the model but also on the maturity of core design systems, formalized standards, and responsibility procedures. What already works today is accelerating partial operations and iterations, especially where parameters and constraints are described strictly enough.

The bridge between “I said it in words” and “the system executed an action” is standardized tool-connection protocols. In recent years, the Model Context Protocol (MCP) has played a notable role as an open specification for connecting external tools and data sources to an LLM-based application in a standard way [6]. Open-source integrations already exist; in particular, implementations for connecting FreeCAD via an MCP add-on and server have been published. Such solutions do not “make CAD smart by themselves,” but they reduce integration costs and improve reproducibility. Their practical meaning is that the model begins controlling external systems through code: generating and transforming documents and presentations, performing operations in a CAD environment, invoking GIS and data-processing functions, connecting weather and traffic capacity services directly or via libraries –

moving “chat” into an engineering-assistant mode embedded in execution loops.

The next layer of practices begins where an organization has geospatial (GIS) projects. By nature, geospatial data are highly formalized: layers, attributes, geometry, topology, and linkage to time and source. This makes them a favorable environment for combining LLMs with machine and deep learning. In practice, managers, engineers, and designers can ask questions in natural language, while the system translates them into a sequence of operations on data and produces results as a table, chart, or report. The “model understands the request, plans steps, and sequentially calls GIS tools” approach is described, for example, in the GeoGPT framework [7]. In a GIS loop, it is useful to distinguish three layers – data access, analytics, and security:

- **Direct access:** The LLM converts a natural-language request into SQL or Pandas code, enabling instant generation of tables or charts (e.g., selecting roads by traffic load coefficient).
- **Analytics:** ML architectures (CNNs for satellite imagery, Graph Neural Networks for road networks) enable analysis of petabytes of data, object detection, and change monitoring.
- **Security:** Risks of “prompt-to-SQL injections” must be considered; strict access control and full operation logging are required.

Geospatial data are also multilayered, dynamic, and often extremely large. Mature engineering practice therefore includes data provenance, quality control, change management, and applying deep learning where it is justified: CNNs for object detection in imagery, and graph neural networks for connected structures and networks.

Natural-language access to databases reduces the barrier of “I don’t know query languages,” because the question can be

asked in plain words. Scientifically and practically, the “natural language – > SQL” direction (Text-to-SQL, NL2SQL) received a strong boost with large language models, as confirmed by a 2024 survey that systematizes approaches, errors, and evaluation methods [8].

But this is precisely where caution is mandatory: a natural-language interface to databases creates new attack surfaces and new classes of errors. Work on “prompt-to-SQL injections” shows that unauthorized or malformed user instructions can lead to vulnerabilities analogous to SQL injection, in systems where the language model itself generates the query [9]. Therefore, mature practice always includes: permission restriction, banning dangerous operations, validating the generated query, executing in a safe perimeter, and mandatory logging.

A natural continuation is computer vision for transportation – the most mature and proven practice in applied road-scene analytics (Fig. 3). Automated recognition of traffic-flow characteristics has accumulated a large body of experience, and many neural architectures are applied successfully. In production, the pipeline is often pragmatic: a field specialist uploads video footage to a server, an office engineer validates it (quality, metadata, match to location and time), then the video files go to automated recognition, followed by traffic intensity estimation, classification, aggregation of indicators, and report generation [10]. The principle “data quality first, then neural networks” sharply improves robustness. In global literature and surveys, this is described as the chain “detection – > tracking – > behavior analysis” [11]. In this context, the LLM does not replace vision models; its role is to manage the processing scenario, explain metrics, generate text reporting, detect inconsistencies, and support error interpretation. A classic risk remains: performance drifts when conditions change (weather, lighting, camera angle, flow density) [12], so any report must include quality control and an error log.

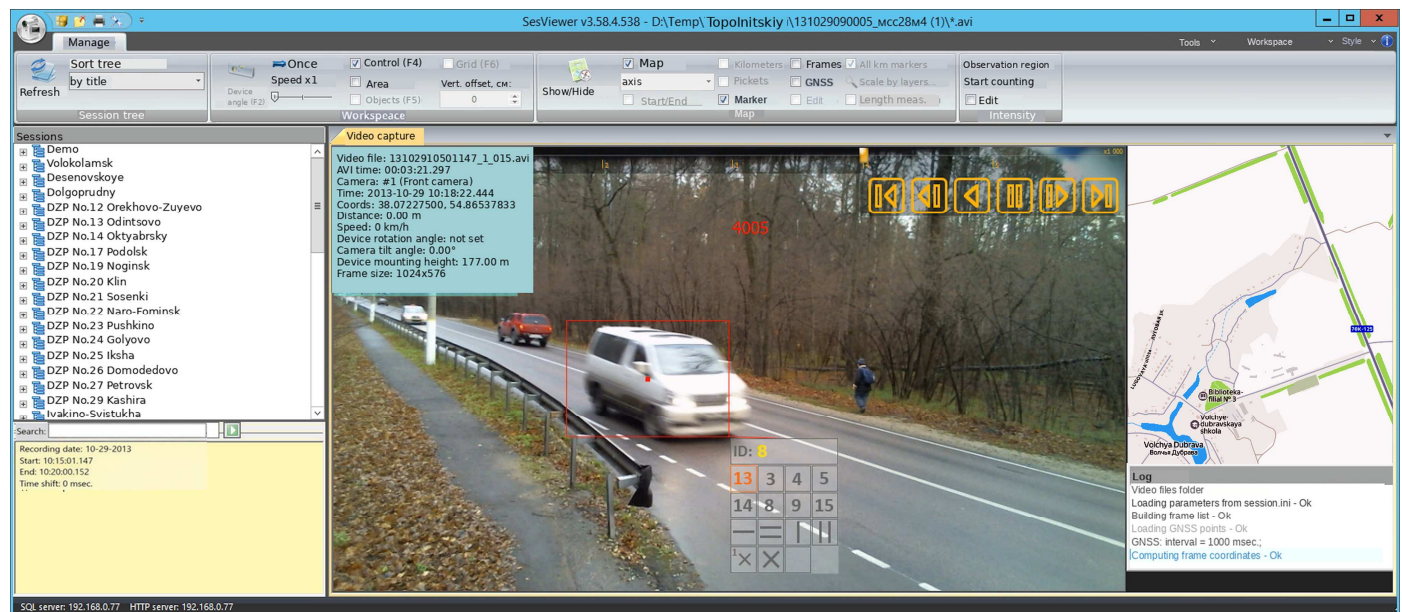


Fig. 3. Computer vision: analysis interface (map/video frame/log)

From computer vision, it is logical to arrive at the idea of an integrated “agent” that collects traffic-recognition data, computes speeds, and generates a report automatically. At the engineering-architecture level, this is no longer “one model,” but a bundle of models and rules where responsibility boundaries are critical: what counts as measurement, what as interpretation, and where manual verification begins.

III. ECONOMICS AND PRAGMATISM

Any discussion of intelligent systems ultimately runs into economics. The cloud is convenient, but you pay for that convenience; local solutions give control, but shift costs into infrastructure, maintenance, and competencies. In modern platforms, cost is determined by tokens (volume of input and output text) and additional tools such as search or index storage. In documentation – for example, Yandex AI Studio (<https://yandex.cloud/en/docs/ai-studio/pricing>) – this is described as token-based pricing for the model and separate pricing for certain operations/services.

For real calculations, it matters that you pay not only for processed text. Typical cloud pricing includes different consumption units: tokens (text volume), “request” (each service call), gigabytes of traffic, “resource×hour” (e.g., network resources), and “gigabyte×hour” for log storage. Total cost typically combines model + tools + infrastructure + storage. Token count depends directly on prompt size: the more instructions, text, and code are placed in context, the higher the spend and cost, and the more strongly model limits matter. Predicting total spend in advance is difficult: it depends on both prompt size and the generated output volume dictated by the request itself. A clear illustration is generating a CAD script: even a “simple” task may require several iterations of refinement and debugging. Cost is not formed by a single query, but by a series of runs – “generate –> test –> refine” – so costs grow multiplicatively. Therefore, economics should be evaluated over the full development cycle, not by “one lucky answer.”

An “engineering” perspective is especially useful here: measure not the price of a single request, but the price of one useful operation – one correct report, one correct data transformation, one verified parameter change. Then cost comparisons become meaningful across tools and robust to pricing changes.

IV. CONCLUSION

When a model produces an undesirable result, “asking harder” usually does not help. What helps is refining expectations until they are verifiable: defining what counts as correct, by what criteria we recognize it, what constraints apply, and what examples are reference standards. In this sense, if an LLM produces a “bad result,” the engineer should first evaluate their own input. A poorly formulated command (off-target or inflated) fails the task just as surely as in a strong

chain of command. Even in local mode, without the internet, the network can be effective because its task is not data retrieval, but management of available tools. For the modern design engineer, AI is not an object of study but a subject – an instrument used to study and transform the world – and it remains useful only when results pass verification by computation, data, and rules.

ACKNOWLEDGMENTS

The article was prepared within the state assignment of Ministry of Science and Higher Education of the Russian Federation (theme No. FFSM-2025-0004).

REFERENCES

- [1] A. Vaswani et al., “Attention Is All You Need,” in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NIPS)*, Long Beach, CA, 2017, pp. 5998–6008.
- [2] A. Sarkar, and I. Drosos, “Vibe coding: programming through conversation with artificial intelligence,” 2025, arXiv:2506.23253, doi: 10.48550/arXiv.2506.23253
- [3] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” 2023, *ICML'23: Proceedings of the 40th International Conference on Machine Learning*, article No.: 1182, pp. 28492–28518.
- [4] J. Zhou, and J. D. Camba, “The status, evolution, and future challenges of multimodal LLMs in parametric CAD (Review),” *Expert Syst. Appl.*, vol. 282, Art. no. 127520, 2025, doi: 10.1016/j.eswa.2025.127520.
- [5] R. Wang, Y. Yuan, S. Sun, and J. Bian, “Text-to-CAD Generation Through Infusing Visual Feedback in Large Language Models,” 2025, arXiv:2501.19054, doi: 10.48550/arXiv.2501.19054.
- [6] H. Li, Y. Xu, and T. Hong, “EnergyPlus-MCP: A model-context-protocol server for ai-driven building energy modeling,” *SoftwareX*, vol. 32, p. 102367, Dec. 2025, doi: 10.1016/j.softx.2025.102367.
- [7] Y. Zhang, C. Wei, Z. He, and W. Yu, “GeoGPT: An assistant for understanding and processing geospatial tasks,” *International Journal of Applied Earth Observation and Geoinformation*, vol. 131, p. 103976, Jul. 2024, doi: 10.1016/j.jag.2024.103976.
- [8] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, “A Survey on Employing Large Language Models for Text-to-SQL Tasks,” *ACM Comput. Surv.*, vol. 58, no. 2, Art. no. 54, pp. 1–37, Sept. 2025, doi: 10.1145/3737873.
- [9] R. Pedro, M. E. Coimbra, D. Castro, P. Carreira, and N. Santos, “Prompt-to-SQL Injections in LLM-Integrated Web Applications,” in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, 2025, pp. 1768–1780, doi: 10.1109/ICSE55347.2025.00007.
- [10] A. Kotov, and P. Pospelov, “Engineering Tools and Methods of Estimation of Traffic Capacity Using Mobile Video Monitoring,” in *Transportation Research Procedia: 12th International Conference “Organization and Traffic Safety Management in Large Cities” (SPbOTSIC 2016)*, vol. 20, St. Petersburg, Russia: Elsevier B.V., 2017, pp. 347–354, doi: 10.1016/j.trpro.2017.01.044.
- [11] W. Zhou et al., “Vision Technologies with Applications in Traffic Surveillance Systems: A Holistic Survey,” *ACM Comput. Surv.*, vol. 58, no. 3, pp. 1–47, 2025, doi: 10.1145/3760525.
- [12] C. Ma, and F. Xue, “A review of vehicle detection methods based on computer vision,” *J. Intell. Connected Veh.*, vol. 7, no. 1, pp. 1–18, 2024, doi: 10.26599/JICV.2023.9210019.